

OpenStack Cloud Portal

Step-by-Step Deployment Plan

Development & Production Environments

Based on: OpenStack Cloud Portal - Platform Deployment Specification v1.0

Document Version:	1.0
Date:	April 09, 2026
Classification:	Confidential
Status:	Draft for Review
Author:	Platform Engineering Team
Reference:	Deployment Spec v1.0

Table of Contents

PART A DEVELOPMENT ENVIRONMENT DEPLOYMENT

1. Prerequisites & Environment Setup
2. Infrastructure Services Setup
3. Build Application Services
4. Deploy with Docker Compose
5. Post-Deployment Verification (Dev)
6. Developer Workflow & Day-2 Operations

PART B PRODUCTION ENVIRONMENT DEPLOYMENT

7. Production Prerequisites & Planning
8. Kubernetes Cluster Provisioning
9. Namespace & RBAC Setup
10. Secrets & Configuration Management
11. Deploy Infrastructure Tier (Prod)
12. Deploy Customer Portal Services
13. Deploy Admin Portal (Isolated)
14. Ingress, TLS & DNS Configuration
15. Horizontal Pod Autoscaler & Scaling
16. Monitoring, Logging & Alerting Setup
17. Security Hardening Checklist
18. BCDR Setup & Validation
19. Production Smoke Tests & Go-Live
20. Post Go-Live Operations & Runbook

PART C APPENDICES

- A. Environment Comparison Matrix
- B. Complete Command Reference
- C. Troubleshooting Guide
- D. Rollback Procedures

PART A

Development Environment Deployment

Docker Compose based local development environment

Single-node, non-HA, developer-friendly setup

1. Prerequisites & Environment Setup

1.1 Hardware Requirements (Development)

Resource	Minimum	Recommended
CPU	4 cores	8 cores
RAM	8 GB	16 GB
Disk	20 GB free	50 GB SSD
OS	macOS / Linux / WSL2	macOS (Apple Silicon) or Linux

1.2 Software Prerequisites

Step 1: Install Docker Desktop (or Docker Engine + Compose)

- macOS: brew install --cask docker
- Linux: Follow official Docker CE install guide for your distro
- Verify: docker --version (requires v20.10+)
- Verify: docker compose version (requires v2.0+)

Step 2: Install Git

- macOS: brew install git
- Linux: sudo apt-get install git (or yum install git)
- Verify: git --version (requires v2.30+)

Step 3: Install Node.js (for local development/debugging)

- Install via nvm: nvm install 18
- Verify: node --version (requires v18+)
- Verify: npm --version (requires v9+)

Step 4: Install Python (for local development/debugging)

- Install via pyenv: pyenv install 3.11
- Verify: python3 --version (requires v3.11+)
- Verify: pip3 --version

Step 5: Install kubectl (optional, for K8s testing)

- macOS: brew install kubectl
- Verify: kubectl version --client

1.3 Clone Repository

Step 6: Clone the project repository

```
git clone <repository-url> cloudsandbox
cd cloudsandbox/openstack-portal
```

1.4 Configure Environment Variables

Step 7: Create and configure .env file

```
cp .env.example .env
```

Edit .env with appropriate values:

Variable	Dev Value	Description	Action Required
OS_AUTH_URL	http://keystone:5000/v3	OpenStack Keystone endpoint	Update if using real OpenStack
OS_USERNAME	admin	OpenStack admin user	Update for your environment
OS_PASSWORD	admin_password	OpenStack admin password	CHANGE to real password
POSTGRES_DB	openstack_portal	Database name	Keep default
POSTGRES_USER	portal_user	Database user	Keep default
POSTGRES_PASSWORD	portal_pass	Database password	OK for dev, CHANGE for staging
JWT_SECRET	your-super-secret...	JWT signing key	CHANGE - use random 64+ chars
REDIS_URL	redis://redis:6379	Redis connection	Keep default
RABBITMQ_URL	amqp://guest:guest@...	RabbitMQ connection	Keep default
SMTP_HOST	smtp.gmail.com	Email server	Configure for email testing
NODE_ENV	development	Runtime mode	Keep as 'development'

WARNING: Never commit .env files to Git. The .gitignore should already exclude .env. Verify before committing.

2. Infrastructure Services Setup

2.1 Start Infrastructure Services First

Infrastructure services (PostgreSQL, Redis, RabbitMQ) must be running and healthy before application services start. Docker Compose handles this via `depends_on` + `healthcheck` conditions.

Step 8: Start infrastructure services only (for verification)

```
docker compose up -d postgres redis rabbitmq
```

Step 9: Verify PostgreSQL is healthy

```
docker compose ps postgres
# Should show 'healthy' status

# Test connection:
docker exec portal-postgres pg_isready -U portal_user -d openstack_portal
# Expected: accepting connections
```

Step 10: Verify Redis is healthy

```
docker compose ps redis

docker exec portal-redis redis-cli ping
# Expected: PONG
```

Step 11: Verify RabbitMQ is healthy

```
docker compose ps rabbitmq

# RabbitMQ Management UI available at:
# http://localhost:15672 (guest/guest)
```

2.2 Database Initialization

Step 12: Verify database extensions are created

- The `setup-db.sh` script runs automatically on first volume creation
- It creates `uuid-oss` and `pgcrypto` extensions
- Application services will auto-create their own tables on startup

```
docker exec portal-postgres psql -U portal_user -d openstack_portal \
-c "SELECT extname FROM pg_extension;"
# Should list: uuid-oss, pgcrypto
```

3. Build Application Services

3.1 Build All Docker Images

Step 13: Build all service images using the build script

```
# Option A: Use the build script
chmod +x scripts/build.sh
./scripts/build.sh

# Option B: Build via docker compose (preferred for dev)
docker compose build
```

This builds 7 images:

Image	Dockerfile	Base Image	Language
openstack-portal/api-gateway	services/api-gateway/Dockerfile	node:18-alpine	Node.js
openstack-portal/auth-service	services/auth-service/Dockerfile	node:18-alpine	Node.js
openstack-portal/admin-service	services/admin-service/Dockerfile	python:3.11-slim	Python
openstack-portal/cloud-service	services/cloud-service/Dockerfile	python:3.11-slim	Python
openstack-portal/billing-service	services/billing-service/Dockerfile	node:18-alpine	Node.js
openstack-portal/notification-service	services/notification-service/Dockerfile	node:18-alpine	Node.js
openstack-portal/frontend	frontend/Dockerfile	node + nginx:alpine	React/TS

Step 14: Verify all images built successfully

```
docker images | grep openstack-portal
# Should show 7 images with 'latest' tag
```

4. Deploy with Docker Compose

4.1 Full Stack Deployment

Step 15: Start all services using dev-start script

```
# Option A: Use the dev-start script (recommended)
chmod +x scripts/dev-start.sh
./scripts/dev-start.sh

# Option B: Start manually
docker compose up --build -d
```

Step 16: Monitor startup and wait for all services to be healthy

```
# Watch all containers in real-time:
docker compose ps

# Wait for all healthchecks (takes ~60 seconds):
watch docker compose ps

# Check logs if any service fails:
docker compose logs <service-name>
```

4.2 Service Startup Order

Docker Compose enforces the following dependency chain:

Phase	Services	Depends On	Wait Condition
Phase 1	postgres, redis, rabbitmq	None	healthcheck: healthy
Phase 2	auth-service	postgres, redis	healthcheck: healthy
Phase 3	admin-service, cloud-service	postgres, rabbitmq	healthcheck: healthy
Phase 4	billing-service	postgres, rabbitmq	healthcheck: healthy
Phase 5	notification-service	rabbitmq	started
Phase 6	api-gateway	auth, admin, cloud, billing	healthcheck: healthy
Phase 7	frontend	api-gateway	started

4.3 Access Points (Development)

Service	URL	Credentials	Purpose
Frontend Portal	http://localhost	admin / admin123	Main web UI
API Gateway	http://localhost:3000	JWT Bearer token	API entry point
Admin Service Docs	http://localhost:8001/docs	N/A	Swagger API docs

Cloud Service Docs

<http://localhost:8002/docs>

N/A

Swagger API docs

RabbitMQ Management

`http://localhost:15672`

guest / guest

Queue monitoring

PostgreSQL

localhost:5433

portal_user / portal_pass

Database (mapped to
5433)

5. Post-Deployment Verification (Dev)

5.1 Health Check Verification

Step 17: Verify all service health endpoints

```
# API Gateway
curl -s http://localhost:3000/health | jq .

# Auth Service (via gateway)
curl -s http://localhost:3000/api/auth/health | jq .

# Admin Service
curl -s http://localhost:8001/health

# Cloud Service
curl -s http://localhost:8002/health

# Billing Service (via gateway)
curl -s http://localhost:3000/api/billing/health | jq .
```

5.2 Functional Smoke Tests

Step 18: Test authentication flow

```
# Login and get JWT token:
curl -s -X POST http://localhost:3000/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"username": "admin", "password": "admin123"}' | jq .

# Save token for subsequent requests:
TOKEN=$(curl -s -X POST http://localhost:3000/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"username": "admin", "password": "admin123"}' | jq -r '.token')
```

Step 19: Test API Gateway routing

```
# Test admin endpoints:
curl -s http://localhost:3000/api/admin/projects \
  -H "Authorization: Bearer $TOKEN" | jq .

# Test cloud service:
curl -s http://localhost:3000/api/cloud/instances \
  -H "Authorization: Bearer $TOKEN" | jq .
```

Step 20: Verify frontend loads correctly

- Open <http://localhost> in a browser
- Verify login page renders
- Login with admin / admin123

- Verify dashboard loads with navigation sidebar
- Navigate to Admin > Projects to verify admin module

5.3 Infrastructure Verification

Step 21: Verify database tables were created

```
docker exec portal-postgres psql -U portal_user -d openstack_portal \  
-c "\dt"  
# Should list tables created by each service
```

Step 22: Verify RabbitMQ queues are declared

```
curl -s -u guest:guest http://localhost:15672/api/queues | jq '.[].name'
```

6. Developer Workflow & Day-2 Operations

6.1 Daily Development Commands

Step 23: Common development commands

```
# Start all services:
docker compose up -d

# Stop all services (preserves data):
docker compose stop

# Restart a specific service after code changes:
docker compose up -d --build <service-name>

# View logs for a specific service:
docker compose logs -f <service-name>

# View all logs:
docker compose logs -f

# Full teardown (destroys data volumes):
docker compose down -v
```

6.2 Database Operations

Step 24: Database management commands

```
# Connect to PostgreSQL shell:
docker exec -it portal-postgres psql -U portal_user -d openstack_portal

# Run Alembic migrations (admin-service):
docker exec portal-admin alembic upgrade head

# Create a new migration:
docker exec portal-admin alembic revision --autogenerate -m "description"

# Reset database (DESTRUCTIVE):
docker compose down -v && docker compose up -d
```

6.3 Troubleshooting Common Issues

Issue	Cause	Solution
Service stays 'starting'	Dependency not healthy	Check depends_on service logs: docker compose logs <dep>
Port already in use	Another process on port	lsof -i :<port> and kill process, or change port in .env
Database connection		

refused

PostgreSQL not ready

docker compose restart postgres; wait for healthcheck

CORS errors in browser

API URL mismatch

Verify VITE_API_BASE_URL in .env matches gateway URL

JWT token expired

24h token TTL

Re-login to get a new token

RabbitMQ not connecting

Service started too early

```
docker compose restart billing-service  
notification-service
```

PART B

Production Environment Deployment

Kubernetes-based, HA, multi-namespace production deployment
Customer Portal + Isolated Admin Portal

7. Production Prerequisites & Planning

7.1 Infrastructure Requirements

Resource	Minimum (Prod)	Recommended (Prod)	Notes
K8s Master Nodes	3	3 (HA control plane)	Across 3 availability zones
K8s Worker Nodes	5	7-10	Separate node pools for app + data
Worker Node CPU	4 vCPU each	8 vCPU each	Total: 20-80 vCPU
Worker Node RAM	16 GB each	32 GB each	Total: 80-320 GB
Storage (SSD)	500 GB	1 TB	For PVs (PostgreSQL, RabbitMQ)
Load Balancer	1 external LB	2 (customer + admin)	L4 or L7
VPN Gateway	1	1 (HA pair)	For admin portal access
Container Registry	1 private registry	Harbor or cloud-native	With vulnerability scanning

7.2 Software Prerequisites

Step 1: Required tools on the deployment workstation

- kubectl v1.27+ (matching cluster version)
- helm v3.12+ (for infrastructure operators)
- docker v24+ (for building images)
- git v2.40+
- openssl (for certificate generation)
- jq (for JSON parsing)
- kustomize v5+ (for manifest overlays)

7.3 Pre-Deployment Approvals

- ☐ Change request approved by Change Advisory Board (CAB)
- ☐ Security review completed and signed off
- ☐ Network architecture reviewed by network team
- ☐ Capacity planning reviewed and approved
- ☐ Disaster recovery plan documented and tested
- ☐ Rollback plan documented and rehearsed
- ☐ On-call team notified of deployment window
- ☐ Customer communication prepared (if applicable)
- ☐ Monitoring dashboards and alerts configured

7.4 Deployment Timeline (Estimated)

Phase	Duration	Owner	Dependencies
K8s Cluster Provisioning	2-4 hours	Infrastructure/SRE	Network, compute resources
Namespace & RBAC Setup	30 min	Platform Engineer	Cluster ready
Secrets & Config Setup	1 hour	Platform + Security	Vault/KMS access
Infrastructure Tier Deploy	1-2 hours	Platform Engineer	Namespaces, secrets
Customer Portal Deploy	1 hour	Platform Engineer	Infrastructure tier healthy
Admin Portal Deploy	1 hour	Platform Engineer	Infrastructure tier healthy
Ingress & TLS Setup	1 hour	Platform + Network	DNS records, cert authority
Monitoring & Alerting	2 hours	SRE/Observability	Services running
Security Hardening	2 hours	Security Engineer	All services deployed
Smoke Tests & Validation	1-2 hours	QA + Platform	Everything deployed
Go-Live Decision	30 min	Engineering Lead	All validations passed

Total estimated duration: 12-16 hours (typically spread across 2 days)

8. Kubernetes Cluster Provisioning

8.1 Cluster Configuration

Step 2: Provision Kubernetes cluster with HA control plane

- Use managed K8s (EKS, AKS, GKE) or kubeadm for bare-metal
- 3 master nodes across 3 availability zones
- Enable Kubernetes audit logging on API server
- Configure etcd encryption at rest

Example cluster configuration (adjust for your provider):

```
# Example: Create cluster (cloud provider specific)
# AWS EKS:
eksctl create cluster --name openstack-portal-prod \
  --region us-east-1 --zones us-east-1a,us-east-1b,us-east-1c \
  --nodegroup-name app-nodes --node-type m5.2xlarge \
  --nodes 5 --nodes-min 3 --nodes-max 10

# OR bare-metal with kubeadm:
kubeadm init --control-plane-endpoint <lb-dns>:6443 \
  --upload-certs --pod-network-cidr=10.244.0.0/16
```

Step 3: Create separate node pools

```
# Application node pool (for microservices):
# Labels: role=application
# Taints: none

# Data node pool (for PostgreSQL, Redis, RabbitMQ):
# Labels: role=data
# Taints: dedicated=data:NoSchedule
# Use SSD-backed storage, higher memory

# Admin node pool (optional, for admin portal):
# Labels: role=admin
```

Step 4: Install required cluster addons

```
# 1. CNI (Calico for network policies):
kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.26/manifests/calico.yaml

# 2. NGINX Ingress Controller:
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm install ingress-nginx ingress-nginx/ingress-nginx \
  --namespace ingress-nginx --create-namespace \
  --set controller.replicaCount=2 \
  --set controller.nodeSelector.role=application

# 3. cert-manager (for TLS certificates):
helm repo add jetstack https://charts.jetstack.io
helm install cert-manager jetstack/cert-manager \
  --namespace cert-manager --create-namespace \
  --set installCRDs=true

# 4. Metrics Server (for HPA):
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
```

Step 5: Verify cluster is ready

```
kubectl get nodes -o wide
# All nodes should be 'Ready'

kubectl get pods -A
# All system pods should be 'Running'

kubectl cluster-info
```

9. Namespace & RBAC Setup

9.1 Create Namespaces

Step 6: Create Customer Portal namespace

```
kubectl apply -f k8s/base/namespace.yaml
# Creates: namespace/openstack-portal
```

Step 7: Create Admin Portal namespace (isolated)

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Namespace
metadata:
  name: openstack-admin
  labels:
    purpose: admin-portal
    isolation: strict
EOF
```

Step 8: Create supporting namespaces

```
# Monitoring namespace:
kubectl create namespace monitoring

# Logging namespace:
kubectl create namespace logging
```

9.2 RBAC Configuration

Step 9: Create service accounts and RBAC roles

```
# Platform engineers - full access to portal namespaces:
cat <<EOF | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: platform-admin
  namespace: openstack-portal
subjects:
- kind: Group
  name: platform-engineers
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: admin
  apiGroup: rbac.authorization.k8s.io
EOF
```

Step 10: Apply network policies for namespace isolation

```
# Default deny all in customer portal namespace:
cat <<EOF | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: openstack-portal
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
EOF

# Default deny all in admin namespace:
cat <<EOF | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: openstack-admin
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
EOF
```

Step 11: Apply allow rules for required communication paths

- Allow ingress to API Gateway from ingress-nginx namespace
- Allow inter-service communication within openstack-portal namespace
- Allow egress to data services (postgres, redis, rabbitmq) ports
- Allow egress to OpenStack API endpoints (cloud-service only)
- Allow DNS egress to kube-system (CoreDNS) for all pods
- Admin namespace: allow ingress ONLY from VPN CIDR range

10. Secrets & Configuration Management

10.1 Production Secrets Setup

WARNING: NEVER use default/example passwords in production. All secrets must be unique, strong (32+ random characters), and stored in an external secrets manager (Vault, AWS Secrets Manager, etc.).

Step 12: Generate production secrets

```
# Generate strong random secrets:
JWT_SECRET=$(openssl rand -hex 32)
JWT_REFRESH_SECRET=$(openssl rand -hex 32)
POSTGRES_PASSWORD=$(openssl rand -base64 24)
RABBITMQ_PASSWORD=$(openssl rand -base64 24)

# Encode for K8s secrets:
echo -n "$JWT_SECRET" | base64
echo -n "$POSTGRES_PASSWORD" | base64
```

Step 13: Create Kubernetes secrets (customer portal)

```
kubectl create secret generic portal-secrets \
  --namespace openstack-portal \
  --from-literal=POSTGRES_USER=portal_user \
  --from-literal=POSTGRES_PASSWORD=$POSTGRES_PASSWORD \
  --from-literal=JWT_SECRET=$JWT_SECRET \
  --from-literal=JWT_REFRESH_SECRET=$JWT_REFRESH_SECRET \
  --from-literal=OS_USERNAME=<openstack-admin-user> \
  --from-literal=OS_PASSWORD=<openstack-admin-password> \
  --from-literal=OS_PROJECT_NAME=admin \
  --from-literal=OS_USER_DOMAIN_NAME=Default \
  --from-literal=OS_PROJECT_DOMAIN_NAME=Default \
  --from-literal=SMTP_USER=<smtp-user> \
  --from-literal=SMTP_PASS=<smtp-password>
```

Step 14: Create Kubernetes secrets (admin portal)

```
# Admin portal uses separate JWT secrets for additional isolation
ADMIN_JWT_SECRET=$(openssl rand -hex 32)

kubectl create secret generic admin-portal-secrets \
  --namespace openstack-admin \
  --from-literal=POSTGRES_USER=admin_portal_user \
  --from-literal=POSTGRES_PASSWORD=$ADMIN_DB_PASSWORD \
  --from-literal=JWT_SECRET=$ADMIN_JWT_SECRET \
  --from-literal=OS_USERNAME=<openstack-admin-user> \
  --from-literal=OS_PASSWORD=<openstack-admin-password>
```

10.2 Production ConfigMaps

Step 15: Apply production ConfigMap (customer portal)

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: ConfigMap
metadata:
  name: portal-config
  namespace: openstack-portal
data:
  POSTGRES_HOST: postgres
  POSTGRES_PORT: "5432"
  POSTGRES_DB: openstack_portal
  REDIS_URL: redis://redis:6379
  RABBITMQ_URL: amqp://portal_mq_user:PASSWORD@rabbitmq:5672
  AUTH_SERVICE_URL: http://auth-service:3001
  ADMIN_SERVICE_URL: http://admin-service:8001
  CLOUD_SERVICE_URL: http://cloud-service:8002
  BILLING_SERVICE_URL: http://billing-service:3002
  NOTIFICATION_SERVICE_URL: http://notification-service:3003
  OS_AUTH_URL: http://<keystone-host>:5000/v3
  OS_REGION_NAME: RegionOne
  OS_INTERFACE: public
  OS_IDENTITY_API_VERSION: "3"
  LOG_LEVEL: info
  NODE_ENV: production
EOF
```

IMPORTANT: Update OS_AUTH_URL to point to your actual OpenStack Keystone endpoint. Update RABBITMQ_URL with the production password.

11. Deploy Infrastructure Tier (Production)

11.1 PostgreSQL HA Deployment

Step 16: Deploy PostgreSQL with HA (recommended: CloudNativePG operator)

```
# Option A: CloudNativePG Operator (RECOMMENDED for production HA)
kubectl apply -f https://raw.githubusercontent.com/cloudnative-pg/cloudnative-pg/release-1.22/releases/cnpg-1.22.

# Wait for operator to be ready:
kubectl wait --for=condition=Available deployment/cnpg-controller-manager \
  -n cnpg-system --timeout=120s
```

Step 17: Create PostgreSQL cluster (3-node HA)

```
cat <<EOF | kubectl apply -f -
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: postgres-cluster
  namespace: openstack-portal
spec:
  instances: 3
  primaryUpdateStrategy: unsupervised
  storage:
    size: 50Gi
    storageClass: gp3-encrypted # Adjust for your cloud
  resources:
    requests: { cpu: 500m, memory: 1Gi }
    limits:   { cpu: 2, memory: 4Gi }
  postgresql:
    parameters:
      max_connections: "200"
      shared_buffers: 512MB
      ssl: "on"
  bootstrap:
    initdb:
      database: openstack_portal
      owner: portal_user
  backup:
    barmanObjectStore:
      destinationPath: s3://<backup-bucket>/postgres/
      s3Credentials:
        accessKeyId:
          name: backup-creds
          key: ACCESS_KEY_ID
        secretAccessKey:
          name: backup-creds
          key: SECRET_ACCESS_KEY
      retentionPolicy: "30d"
EOF
```

Step 18: Verify PostgreSQL cluster is healthy

```
kubectl get cluster postgres-cluster -n openstack-portal
# STATUS should be 'Cluster in healthy state'

kubectl get pods -l cnpg.io/cluster=postgres-cluster -n openstack-portal
# 3 pods: 1 primary + 2 replicas, all Running
```

ALTERNATIVE: If not using CloudNativePG, deploy using the existing k8s/base/postgres.yaml for a single-node setup, then configure Patroni or Stolon for HA separately.

11.2 Redis Deployment

Step 19: Deploy Redis (HA with Sentinel for production)

```
# Option A: Redis Sentinel via Bitnami Helm chart (recommended):
helm repo add bitnami https://charts.bitnami.com/bitnami
helm install redis bitnami/redis \
  --namespace openstack-portal \
  --set sentinel.enabled=true \
  --set sentinel.replicas=3 \
  --set replica.replicaCount=3 \
  --set auth.enabled=true \
  --set auth.password=$REDIS_PASSWORD \
  --set persistence.size=10Gi

# Option B: Single-node (from existing manifest - NOT recommended for prod):
kubectl apply -f k8s/base/redis.yaml
```

Step 20: Verify Redis is operational

```
kubectl get pods -l app.kubernetes.io/name=redis -n openstack-portal
# All Redis pods should be Running
```

11.3 RabbitMQ Deployment

Step 21: Deploy RabbitMQ (cluster for production)

```
# Option A: RabbitMQ Cluster Operator (recommended):
kubectl apply -f https://github.com/rabbitmq/cluster-operator/releases/latest/download/cluster-operator.yml

cat <<EOF | kubectl apply -f -
apiVersion: rabbitmq.com/v1beta1
kind: RabbitmqCluster
metadata:
  name: rabbitmq
  namespace: openstack-portal
spec:
  replicas: 3
  resources:
    requests: { cpu: 500m, memory: 1Gi }
    limits:   { cpu: 2, memory: 2Gi }
  persistence:
    storageClassName: gp3-encrypted
    storage: 20Gi
  rabbitmq:
    additionalConfig: |
      default_user = portal_mq_user
      default_pass = CHANGE_ME_STRONG_PASSWORD
EOF

# Option B: Single-node (from existing manifest):
kubectl apply -f k8s/base/rabbitmq.yaml
```

Step 22: Verify RabbitMQ cluster

```
kubectl get pods -l app.kubernetes.io/name=rabbitmq -n openstack-portal
# 3 pods, all Running

kubectl exec rabbitmq-server-0 -n openstack-portal -- rabbitmqctl cluster_status
```

12. Deploy Customer Portal Services

12.1 Build & Push Container Images

Step 23: Tag and push images to private registry

```
REGISTRY=registry.example.com/openstack-portal
TAG=v1.0.0 # Use semantic versioning

# Build all images:
REGISTRY=$REGISTRY TAG=$TAG ./scripts/build.sh

# Push all images:
for svc in api-gateway auth-service admin-service cloud-service \
    billing-service notification-service frontend; do
    docker push $REGISTRY/$svc:$TAG
done
```

Step 24: Update image tags in Kubernetes manifests

```
# Use kustomize to override image tags for production:
# k8s/overlays/production/kustomization.yaml

# Or update services.yaml directly:
sed -i 's|openstack-portal/|registry.example.com/openstack-portal/|g' \
    k8s/base/services.yaml
sed -i 's|:latest|:v1.0.0|g' k8s/base/services.yaml
```

12.2 Deploy Application Services

Step 25: Deploy all customer portal services

```
# Apply all service deployments and K8s services:
kubectl apply -f k8s/base/services.yaml

# Wait for all deployments to roll out:
kubectl rollout status deployment/api-gateway -n openstack-portal --timeout=120s
kubectl rollout status deployment/auth-service -n openstack-portal --timeout=120s
kubectl rollout status deployment/admin-service -n openstack-portal --timeout=120s
kubectl rollout status deployment/cloud-service -n openstack-portal --timeout=120s
kubectl rollout status deployment/billing-service -n openstack-portal --timeout=120s
kubectl rollout status deployment/notification-service -n openstack-portal --timeout=120s
kubectl rollout status deployment/frontend -n openstack-portal --timeout=120s
```

Step 26: Verify all pods are Running and Ready

```
kubectl get pods -n openstack-portal -o wide
```

Expected: All pods in 'Running' state with READY x/x

# NAME	READY	STATUS	RESTARTS
# api-gateway-xxxxx-xxxxx	1/1	Running	0
# auth-service-xxxxx-xxxxx	1/1	Running	0
# admin-service-xxxxx-xxxxx	1/1	Running	0
# cloud-service-xxxxx-xxxxx	1/1	Running	0
# billing-service-xxxxx-xxxxx	1/1	Running	0
# notification-service-xxxxx-xxxxx	1/1	Running	0
# frontend-xxxxx-xxxxx	1/1	Running	0

Step 27: Verify service endpoints

```
kubectl get svc -n openstack-portal
```

Test health endpoints from within cluster:

```
kubectl run -it --rm debug --image=curlimages/curl \
  -n openstack-portal -- curl -s http://api-gateway:3000/health
```

13. Deploy Admin Portal (Isolated)

WARNING: The Admin Portal MUST be deployed in a separate namespace with VPN-only access. It must NEVER be exposed to the public internet. This is a critical security requirement per the Deployment Specification.

13.1 Admin Namespace Configuration

Step 28: Apply admin-specific ConfigMap

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: ConfigMap
metadata:
  name: admin-portal-config
  namespace: openstack-admin
data:
  POSTGRES_HOST: postgres-cluster-rw.openstack-portal.svc.cluster.local
  POSTGRES_PORT: "5432"
  POSTGRES_DB: admin_portal_db
  REDIS_URL: redis://redis-master.openstack-portal.svc.cluster.local:6379
  ADMIN_SERVICE_URL: http://admin-service:8001
  AUTH_SERVICE_URL: http://admin-auth-service:3001
  OS_AUTH_URL: http://<keystone-host>:5000/v3
  LOG_LEVEL: info
  NODE_ENV: production
  SESSION_TIMEOUT: "28800"
  IDLE_TIMEOUT: "1800"
  MFA_REQUIRED: "true"
EOF
```

13.2 Admin Service Deployments

Step 29: Deploy Admin API Gateway

```

cat <<EOF | kubectl apply -f -
apiVersion: apps/v1
kind: Deployment
metadata:
  name: admin-api-gateway
  namespace: openstack-admin
spec:
  replicas: 2
  selector:
    matchLabels: { app: admin-api-gateway }
  template:
    metadata:
      labels: { app: admin-api-gateway }
    spec:
      containers:
      - name: api-gateway
        image: $REGISTRY/api-gateway:$TAG
        ports: [{ containerPort: 3000 }]
        envFrom:
        - configMapRef: { name: admin-portal-config }
        - secretRef: { name: admin-portal-secrets }
        env:
        - { name: ADMIN_MODE, value: "true" }
        - { name: RATE_LIMIT_MAX, value: "50" }
        resources:
          requests: { cpu: 100m, memory: 128Mi }
          limits: { cpu: 500m, memory: 512Mi }
        livenessProbe:
          httpGet: { path: /health, port: 3000 }
          initialDelaySeconds: 10
          periodSeconds: 15
      ---
apiVersion: v1
kind: Service
metadata:
  name: admin-api-gateway
  namespace: openstack-admin
spec:
  selector: { app: admin-api-gateway }
  ports: [{ port: 3000, targetPort: 3000 }]
EOF

```

Step 30: Deploy Admin Service, Admin Auth Service, and Admin Frontend

- Deploy admin-service in openstack-admin namespace (same image, admin-only config)
- Deploy admin-auth-service with MFA_REQUIRED=true and shorter session TTL (8h)
- Deploy admin-frontend with admin-only routes and VPN access banner
- All deployments: 2 replicas minimum, anti-affinity across zones

13.3 Admin Network Policies

Step 31: Apply strict network policies for admin namespace

```

cat <<EOF | kubectl apply -f -
# Allow ingress ONLY from VPN CIDR
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: admin-ingress-from-vpn
  namespace: openstack-admin
spec:
  podSelector: {}
  policyTypes: [Ingress]
  ingress:
  - from:
    - ipBlock:
        cidr: 10.100.0.0/16    # Your VPN CIDR
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: ingress-nginx
  ---
# Allow egress to data services in portal namespace
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: admin-egress-to-data
  namespace: openstack-admin
spec:
  podSelector: {}
  policyTypes: [Egress]
  egress:
  - to:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: openstack-portal
    ports:
    - { port: 5432, protocol: TCP }
    - { port: 6379, protocol: TCP }
  - to: # Allow DNS
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: kube-system
    ports:
    - { port: 53, protocol: UDP }
    - { port: 53, protocol: TCP }
EOF

```

Step 32: Verify admin portal isolation

- From a pod in openstack-portal namespace, attempt to curl admin-api-gateway.openstack-admin:3000 - should FAIL
- From VPN, curl admin.internal.example.com - should SUCCEED
- From public internet, attempt to access admin.internal.example.com - should FAIL (DNS should not resolve)

14. Ingress, TLS & DNS Configuration

14.1 TLS Certificate Setup

Step 33: Configure cert-manager ClusterIssuer

```
cat <<EOF | kubectl apply -f -
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-prod
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: platform-team@example.com
    privateKeySecretRef:
      name: letsencrypt-prod
    solvers:
    - http01:
        ingress:
          class: nginx
EOF
```

14.2 Customer Portal Ingress

Step 34: Apply production ingress for customer portal

```
cat <<EOF | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: portal-ingress
  namespace: openstack-portal
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod
    nginx.ingress.kubernetes.io/proxy-body-size: 100m
    nginx.ingress.kubernetes.io/proxy-read-timeout: "300"
    nginx.ingress.kubernetes.io/rate-limit: "100"
    nginx.ingress.kubernetes.io/rate-limit-window: 1m
    nginx.ingress.kubernetes.io/enable-modsecurity: "true"
spec:
  ingressClassName: nginx
  tls:
  - hosts: [portal.example.com]
    secretName: portal-tls
  rules:
  - host: portal.example.com
    http:
      paths:
      - path: /api
        pathType: Prefix
        backend:
          service: { name: api-gateway, port: { number: 3000 } }
      - path: /
        pathType: Prefix
        backend:
          service: { name: frontend, port: { number: 80 } }
EOF
```

14.3 Admin Portal Ingress (Internal Only)

Step 35: Apply admin ingress (VPN-only, separate ingress class)

```
cat <<EOF | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: admin-ingress
  namespace: openstack-admin
  annotations:
    nginx.ingress.kubernetes.io/whitelist-source-range: 10.100.0.0/16
    cert-manager.io/cluster-issuer: internal-ca
spec:
  ingressClassName: nginx-internal
  tls:
  - hosts: [admin.internal.example.com]
    secretName: admin-tls
  rules:
  - host: admin.internal.example.com
    http:
      paths:
      - path: /api
        pathType: Prefix
        backend:
          service: { name: admin-api-gateway, port: { number: 3000 } }
      - path: /
        pathType: Prefix
        backend:
          service: { name: admin-frontend, port: { number: 80 } }
EOF
```

14.4 DNS Configuration

Step 36: Configure DNS records

- portal.example.com -> A record pointing to external LB IP (customer portal)
- admin.internal.example.com -> A record in INTERNAL DNS ONLY, pointing to internal LB IP
- Verify: nslookup portal.example.com (should resolve from internet)
- Verify: nslookup admin.internal.example.com (should resolve ONLY from VPN/internal network)

15. Horizontal Pod Autoscaler & Scaling

Step 37: Apply HPA configurations

```
kubectl apply -f k8s/base/hpa.yaml
```

Verify HPAs:

```
kubectl get hpa -n openstack-portal
```

Expected output:

# NAME	REFERENCE	TARGETS	MINPODS	MAXPODS
# api-gateway-hpa	Deployment/api-gateway	<x>/70%	2	10
# auth-service-hpa	Deployment/auth-service	<x>/70%	2	8
# cloud-service-hpa	Deployment/cloud-service	<x>/70%	2	8
# frontend-hpa	Deployment/frontend	<x>/70%	2	6

Step 38: Apply admin portal HPA

```
cat <<EOF | kubectl apply -f -
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: admin-api-gateway-hpa
  namespace: openstack-admin
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: admin-api-gateway
  minReplicas: 2
  maxReplicas: 4
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
EOF
```

Step 39: Test scaling behavior

- Verify metrics server is providing data: `kubectl top pods -n openstack-portal`
- Simulate load and verify HPA scales up appropriately
- Verify pods are spread across availability zones
- Manual scaling fallback: `./scripts/scale.sh api-gateway 4`

16. Monitoring, Logging & Alerting Setup

16.1 Deploy Monitoring Stack

Step 40: Install Prometheus + Grafana via kube-prometheus-stack

```
helm repo add prometheus-community \
  https://prometheus-community.github.io/helm-charts

helm install monitoring prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --set prometheus.prometheusSpec.retention=30d \
  --set prometheus.prometheusSpec.storageSpec.volumeClaimTemplate.spec.resources.requests.storage=100Gi \
  --set grafana.adminPassword=<CHANGE_ME>
```

Step 41: Configure ServiceMonitors for application services

- Create ServiceMonitor resources for each microservice
- Ensure metrics endpoints are scraped (e.g., /metrics on each service)
- Import pre-built Grafana dashboards for Node.js and FastAPI

16.2 Deploy Logging Stack

Step 42: Install Loki + Promtail (or EFK stack)

```
helm repo add grafana https://grafana.github.io/helm-charts

helm install loki grafana/loki-stack \
  --namespace logging \
  --set promtail.enabled=true \
  --set loki.persistence.enabled=true \
  --set loki.persistence.size=50Gi
```

16.3 Configure Alerts

Step 43: Set up critical alerting rules

- P1: Any service down for > 5 minutes -> PagerDuty
- P1: PostgreSQL replication lag > 10 seconds -> PagerDuty
- P2: API error rate > 1% for 5 minutes -> Slack
- P2: Pod restart count > 3 in 10 minutes -> Slack
- P3: CPU > 80% sustained for 15 minutes -> Email
- P3: Disk usage > 85% -> Email
- P4: Certificate expiring in < 14 days -> Ticket

Step 44: Verify monitoring is operational

- Access Grafana dashboard and verify data is flowing

- Trigger a test alert and verify it reaches the correct channel
- Verify log aggregation is capturing logs from all services

17. Security Hardening Checklist

Execute each item and check off upon completion:

17.1 Authentication & Access

- ☐ JWT secrets are unique, 64+ character random strings
- ☐ Admin portal requires MFA (TOTP or FIDO2)
- ☐ Admin sessions expire after 8 hours (idle: 30 min)
- ☐ Customer sessions expire after 24 hours
- ☐ Token blacklisting is active in Redis
- ☐ Default credentials (admin/admin123) are CHANGED

17.2 Network Security

- ☐ Network policies applied and tested in all namespaces
- ☐ Admin portal accessible ONLY via VPN
- ☐ TLS 1.3 enforced on all external endpoints
- ☐ mTLS enabled for inter-service communication (Istio/Linkerd)
- ☐ WAF rules active on ingress (ModSecurity / cloud WAF)
- ☐ Egress restricted - only required destinations allowed

17.3 Data Security

- ☐ PostgreSQL volumes use encrypted storage class
- ☐ PostgreSQL connections require SSL (sslmode=verify-full)
- ☐ All secrets in Vault/KMS (not plaintext in ConfigMaps)
- ☐ Separate DB credentials per service (least privilege)
- ☐ PII encrypted at application level
- ☐ Log scrubbing for credentials/tokens active

17.4 Container Security

- ☐ All images scanned - no CRITICAL CVEs
- ☐ Containers run as non-root user
- ☐ Read-only root filesystem enabled
- ☐ Pod Security Standards set to 'restricted'
- ☐ Resource limits enforced on all pods
- ☐ Image pull policy set to 'Always' with signed images

18. BCDR Setup & Validation

18.1 Backup Configuration

Step 45: Configure automated backups

```
# 1. PostgreSQL backups (handled by CloudNativePG if used)
# Verify scheduled backups:
kubectl get scheduledbackup -n openstack-portal

# 2. Install Velero for K8s cluster backup:
velero install --provider aws --bucket <backup-bucket> \
  --secret-file ./credentials-velero \
  --backup-location-config region=us-east-1

# 3. Create daily backup schedule:
velero schedule create daily-backup --schedule='0 2 * * *' \
  --include-namespaces openstack-portal,openstack-admin \
  --ttl 720h
```

Step 46: Verify backup & restore works

```
# Trigger a manual backup:
velero backup create manual-test-backup \
  --include-namespaces openstack-portal

# Verify backup completed:
velero backup describe manual-test-backup

# Test restore to a different namespace (non-destructive):
velero restore create test-restore --from-backup manual-test-backup \
  --namespace-mappings openstack-portal:restore-test
```

18.2 DR Readiness

Step 47: Validate DR procedures

- Verify PostgreSQL replication to DR site is active and lag < 5 seconds
- Verify container images are replicated to DR registry
- Verify DNS failover configuration is in place
- Run a tabletop DR exercise with the team
- Document recovery runbooks and verify they are up to date

19. Production Smoke Tests & Go-Live

19.1 Customer Portal Smoke Tests

Step 48: Execute production smoke test suite

#	Test	Command / Action	Expected Result
1	Portal loads	curl -sI https://portal.example.com	HTTP 200, TLS 1.3
2	API health	curl -s https://portal.example.com/api/health	{ status: ok }
3	Login works	POST /api/auth/login with valid creds	JWT token returned
4	Dashboard loads	Login via browser, navigate to /dashboard	Dashboard renders with data
5	Cloud ops	Create a test instance via portal	Instance created in OpenStack
6	Billing visible	Navigate to Billing > Overview	Usage data displayed
7	Notifications	Trigger action that sends email	Email received

19.2 Admin Portal Smoke Tests

Step 49: Execute admin portal smoke tests (from VPN)

#	Test	Action	Expected Result
1	VPN access only	Access admin URL from public internet	Connection refused / timeout
2	Admin loads	Access admin URL from VPN	Login page renders
3	MFA required	Login with username/password only	MFA prompt appears
4	Admin login	Complete MFA flow	Admin dashboard loads
5	Projects	Navigate to Admin > Projects	Project list displayed
6	Quotas	View/edit a project quota	Quota changes saved
7	Audit log	Navigate to Admin > Audit Log	Recent actions displayed

19.3 Go-Live Checklist

- ☐ All smoke tests passed
- ☐ Monitoring dashboards showing green / healthy
- ☐ Alerting verified (test alert received)
- ☐ Backup schedule active and verified
- ☐ Security hardening checklist completed
- ☐ Network policies verified
- ☐ Admin portal isolation confirmed
- ☐ DNS records propagated and verified

- ☐ TLS certificates valid (not self-signed)
- ☐ On-call rotation established
- ☐ Runbooks accessible to ops team
- ☐ Rollback plan rehearsed
- ☐ Go-Live approval from Engineering Lead: _____
- ☐ Go-Live approval from Security: _____
- ☐ Go-Live date/time: _____

20. Post Go-Live Operations & Runbook

20.1 Day-1 Post Go-Live

- Monitor dashboards intensively for first 4 hours
- Keep rollback capability warm (do not clean up old deployment for 24h)
- Watch for: error rate spikes, latency increases, pod restarts
- On-call engineer available for immediate response

20.2 Ongoing Operations

Task	Frequency	Owner	Procedure
Certificate rotation check	Weekly	SRE	Verify cert-manager renewals
Security patching	Weekly	SRE + Sec	Scan images, update base images
Backup verification	Monthly	SRE	Restore test to verify backup integrity
DR failover test	Semi-annually	SRE + Eng	Controlled failover to DR site
Access review	Quarterly	Security	Review RBAC, VPN access, admin accounts
Capacity review	Quarterly	SRE + Eng	Review HPA metrics, plan scaling
Dependency updates	Monthly	Engineering	Update npm/pip packages, scan for CVEs
Runbook review	Quarterly	SRE	Verify procedures are current

20.3 Scaling Operations

```
# Manual scaling (if HPA is not sufficient):
./scripts/scale.sh api-gateway 6
./scripts/scale.sh cloud-service 5

# View current scaling status:
kubectl get hpa -n openstack-portal
kubectl top pods -n openstack-portal
```

20.4 Rollback Procedure

Step Emergency Rollback: If critical issues are discovered post-deploy

```
# Option 1: Rollback specific deployment to previous revision:
kubectl rollout undo deployment/<service-name> -n openstack-portal

# Option 2: Rollback ALL services to a known-good state:
for svc in api-gateway auth-service admin-service cloud-service \
    billing-service notification-service frontend; do
    kubectl rollout undo deployment/$svc -n openstack-portal
done

# Option 3: Full cluster restore from Velero backup:
velero restore create emergency-restore \
    --from-backup <last-known-good-backup>

# Verify rollback:
kubectl get pods -n openstack-portal
curl -s https://portal.example.com/api/health
```

PART C

Appendices

A. Environment Comparison Matrix

Aspect	Development	Production
Deployment Tool	Docker Compose	Kubernetes (K8s)
Orchestration	docker compose up	kubectl / ArgoCD
Replicas (services)	1 per service	2-10 per service (HPA)
PostgreSQL	Single container, no HA	3-node cluster (CloudNativePG)
Redis	Single container	3-node Sentinel cluster
RabbitMQ	Single container	3-node quorum cluster
TLS	None (HTTP only)	TLS 1.3 (cert-manager)
mTLS	None	Istio service mesh
WAF	None	ModSecurity on ingress
Network Policies	Docker bridge network	Calico NetworkPolicies
Secrets	.env file	K8s Secrets + Vault/KMS
Admin Portal	Same compose stack	Separate namespace + VPN
Monitoring	Docker logs	Prometheus + Grafana + Loki
Backups	None (ephemeral)	Automated daily (Velero + WAL)
DR	None	Hot/Warm standby + DR site
Scaling	Manual container restart	HPA + Cluster Autoscaler
Registry	Local images	Private registry (Harbor)
CI/CD	Manual build	GitHub Actions + ArgoCD
Access Control	Open localhost	RBAC + VPN + MFA (admin)

B. Complete Command Reference

B.1 Development Commands

```
# Start everything:      ./scripts/dev-start.sh
# Build all images:     ./scripts/build.sh
# Stop (keep data):     docker compose stop
# Start again:          docker compose up -d
# Full teardown:        docker compose down -v
# Rebuild one service:  docker compose up -d --build <svc>
# View logs:            docker compose logs -f <svc>
# DB shell:             docker exec -it portal-postgres psql -U portal_user -d openstack_portal
```

B.2 Production Commands

```
# Deploy:               ./scripts/deploy.sh
# Scale service:         ./scripts/scale.sh <svc> <replicas>
# Check pods:           kubectl get pods -n openstack-portal
# Check services:       kubectl get svc -n openstack-portal
# View logs:            kubectl logs -f deployment/<svc> -n openstack-portal
# Rollback:             kubectl rollout undo deployment/<svc> -n openstack-portal
# Check HPA:            kubectl get hpa -n openstack-portal
# Resource usage:       kubectl top pods -n openstack-portal
# Describe pod:         kubectl describe pod <pod-name> -n openstack-portal
# Admin namespace:      kubectl get pods -n openstack-admin
```

C. Troubleshooting Guide

Symptom	Diagnosis	Resolution
Pod in CrashLoopBackOff	kubectl logs <pod> -n openstack-portal openstack-portal --previous	Check env vars, secrets, DB connectivity
Pod stuck in Pending	kubectl describe pod <pod>	Insufficient resources - scale node pool or adjust requests
Ingress not routing	kubectl describe ingress -n openstack-portal	Verify ingress class, service names, port numbers
TLS certificate error	kubectl describe certificate -n openstack-portal	Check cert-manager logs, DNS propagation, ClusterIssuer
DB connection refused	kubectl logs postgres-cluster-1 -n openstack-portal	Check PG is ready, secrets match, network policies allow 5432
HPA not scaling	kubectl describe hpa <hpa-name>	Verify metrics-server is running and providing data
Admin portal unreachable	Check VPN connection, then kubectl get pods -n openstack-admin	Verify VPN is connected, network policies allow VPN CIDR
High API latency	Check Grafana dashboards for bottleneck	Scale service via HPA/manual, check DB query performance
RabbitMQ queue backlog	kubectl exec rabbitmq-0 -- rabbitmqctl list_queues	Scale notification-service, check consumer health

D. Rollback Procedures

Scenario	Procedure	RTO	Impact
Bad code deploy	kubectl rollout undo deployment/<svc>	< 2 min	Zero (rolling update)
Config error	kubectl rollout undo; fix ConfigMap; re-deploy	< 5 min	Brief service disruption
Bad DB migration	Alembic downgrade; rollout undo	< 15 min	Brief service disruption
Cluster-wide issue	Velero restore from last backup	< 1 hour	Data loss up to RPO
Full site failure	DR failover via DNS switch	< 15 min	Brief global disruption

Document Revision History

Version	Date	Author	Changes
1.0	2026-04-09	Platform Engineering	Initial deployment plan